

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Monitoreo del estado de la batería con BroadcastReceiver en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	04	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	25/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Hanco Mullisaca Sergio Danilo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Ejercicio 1: Monitoreo de batería con BroadcastReceiver en Jetpack Compose Enunciado Implementar una pantalla en Jetpack Compose que muestre el nivel de batería en tiempo real mediante un BroadcastReceiver registrado dinámicamente con DisposableEffect. Video de referencia: Broadcast Receivers en Android con Jetpack Compose (YouTube). Desarrollo 1. Implementación del Composable BatteryScreen Se define el componente composable encargado de escuchar los cambios en el estado de la batería mediante un BroadcastReceiver acoplado al ciclo de vida de Compose: <pre> 1 @Composable 2 fun BatteryScreen(modifier: Modifier = Modifier) { 3 var porcentaje by remember { mutableIntStateOf(0) } 4 val context = LocalContext.current 5 6 DisposableEffect(Unit) { 7 val receiver = object : BroadcastReceiver() { 8 override fun onReceive(context: Context?, intent: Intent?) { 9 val nivel = intent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1 10 val escala = intent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1 11 if (nivel != -1 && escala != -1) { 12 porcentaje = (nivel * 100) / escala 13 } 14 } 15 } 16 17 context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED)) 18 Log.d("BatteryScreen", "Receiver registrado") 19 } </pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

20     onDispose {
21         context.unregisterReceiver(receiver)
22         Log.d("BatteryScreen", "Receiver desregistrado")
23     }
24 }
25
26 Text(
27     text = "Batería: $porcentaje%",
28     modifier = modifier
29 )
30 }

```

Análisis: El composable gestiona el estado reactivo con `mutableIntStateOf(0)` y emplea `DisposableEffect` para registrar dinámicamente el `BroadcastReceiver` con `ACTION_BATTERY_CHANGED`, calculando el porcentaje a partir de `EXTRA_LEVEL` y `EXTRA_SCALE`, y garantizando la liberación del recurso en `onDispose` para evitar fugas de memoria.

2. Configuración en MainActivity.kt

Se establece la pantalla principal dentro de la actividad mediante el contenedor `Scaffold`:

```

1 class MainActivity : ComponentActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContent {
5             BatteryMonitor_ComposeTheme {
6                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
7                     BatteryScreen(modifier = Modifier.padding(innerPadding))
8                 }
9             }
10        }
11    }
12 }

```

Análisis: Se define la actividad principal estructurando la vista con `Scaffold`, propagando los márgenes de `innerPadding` hacia `BatteryScreen` mediante `Modifier.padding` para respetar las áreas seguras de la pantalla y asegurar una correcta integración con Material Design.

3. Pruebas y Verificación

- Simulación: Desde Extended Controls > Battery en el emulador, se modificó el nivel de batería, verificando la reactividad inmediata del Text en pantalla.
- Logcat: Se filtró por la etiqueta `BatteryScreen`, corroborando los logs de registro al iniciar y de desregistro al desmontar la vista.

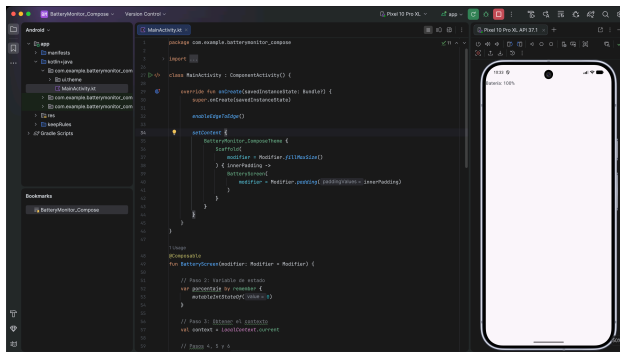


Figura 1: Ejecución inicial en Android Studio.

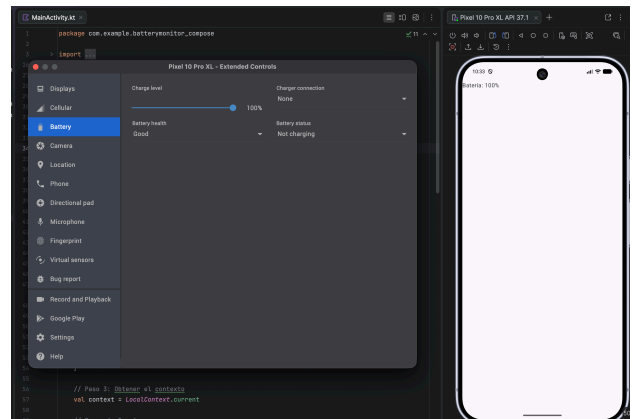


Figura 2: Simulación de batería al 100%.

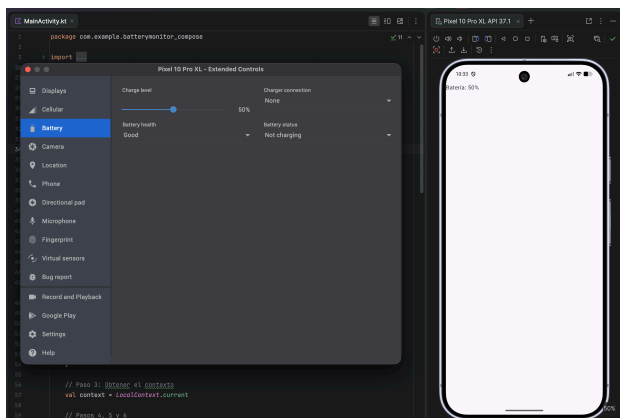


Figura 3: Simulación de batería al 50%.

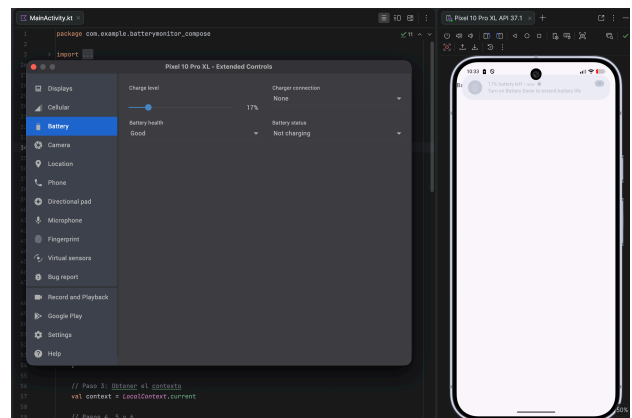


Figura 4: Simulación de batería al 17%.

Modificaciones para Warnings y Buenas Prácticas

- Manejo de `innerPadding`: Se aplicó el `innerPadding` provisto por `Scaffold` para evitar superposiciones con barras del sistema y prevenir advertencias de layout en Compose.
- Parámetro `Modifier`: Se añadió `modifier: Modifier = Modifier` a `BatteryScreen()` para garantizar su reusabilidad y cumplir con las directrices oficiales de Android.
- Optimización con `mutableIntStateOf`: Se sustituyó `mutableStateOf(0)` por `mutableIntStateOf(0)` para optimizar el rendimiento y evitar el boxing innecesario de enteros.
- Validación y Lint: Se comprobó la compilación, pruebas y análisis estático con Android Lint, asegurando un código limpio, funcional y libre de errores.

Ejercicio 2

Enunciado

Instrucciones del segundo ejercicio.

Desarrollo

Solución desarrollada por el integrante encargado.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

Ejercicio 3

Enunciado

Instrucciones del tercer ejercicio.

Desarrollo

Solución desarrollada por el integrante encargado.

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1

Respuesta de la primera pregunta, redactada por el integrante encargado.

Pregunta 2

Respuesta de la segunda pregunta, redactada por el integrante encargado.

Pregunta 3

Respuesta de la tercera pregunta, redactada por el integrante encargado.

III. CONCLUSIONES

- Redacta aquí la primera conclusión de la práctica.
- Redacta aquí la segunda conclusión de la práctica.
- Redacta aquí la tercera conclusión de la práctica.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] I. Sommerville, *Ingeniería del Software*, 9.^a ed. Madrid: Pearson Educación, 2011.
- [2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.^a ed. México: McGraw-Hill, 2010.
- [3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.